

Kutan Arcan Akgül

Game Developer / Software Engineer (Unity, C#)

İstanbul, Türkiye | kutanarcanakgul@gmail.com | +90 553 270 46 86

linkedin.com/in/kutanarcanakgul | github.com/Kutanarcan | kutanarcanakgul.itch.io

Summary

Game developer with **six years of experience** building mobile games in **Unity and C#**. My work rests on **OOP, design patterns** and **clean, modular code**: game rules stay separate from what the player sees, so features can be changed, tested or replaced on their own. I **design, implement and test game features** end to end, review every change with my team, and judge the result by the **user experience** it delivers.

Experience

Game Developer / Software Engineer — Sciplay Turkey

01/2025 – 07/2026

Match Hotel (live-ops triple-match game) · İstanbul, Türkiye · [Google Play](#)

- Designed, implemented and tested two live-ops events end to end — Arthur's Deal and Vitamin Bar — owning **logic, data, UI and analytics**.
- Designed Vitamin Bar around player freedom. Every rule lives in the **logic layer**; views only report input. A player can place a piece, close the popup mid-animation, reopen it and carry on instantly. The architecture exists to serve that **user experience**.
- Reviewed every change with my team lead in **code review** sessions, tested and fixed alongside **QA**, and shaped event design and content with **Product and Art**.
- Built **remote config** structures for **A/B testing** on an epoch-based time system, replacing weekly manual event entry with looping time windows that schedule correctly worldwide.
- Wrote the game's **audio system** from scratch and built the **analytics layer** behind nearly all live events, giving the data team the economy tracking it needed.

Game Developer — Banba Games

08/2022 – 01/2025

Car Racing Simulator: X Street (live open-world racer) · İstanbul, Türkiye · [Google Play](#)

- Rewrote the vehicle **tire-trail mesh system** from scratch: **multithreaded**, writing straight to index and vertex buffers, bypassing Unity's Mesh class. **5 ms to 0.1 ms** per update, **~200 KB to zero** allocation.
- Built the title's **scalable backend infrastructure** and **client-server communication** from scratch, including the sync layer that reconciles offline play with the player's server-side state.
- Integrated the Google Play SDK suite, set up and ran the ad service in production, and owned the **Android build pipeline** end to end, from custom Gradle setup to Play Console releases.
- Built an online **physics-based multiplayer** party game prototype on **FishNet** (Bridge Race combined with Fall Guys-style mechanics) that stayed playable in testing at latencies up to 100 ms.

Game Developer — Talus Studio

08/2021 – 08/2022

Hyper-casual games · İstanbul, Türkiye · studio closed in 2022, titles no longer live · [Gameplay reel](#)

- Shipped **15 hyper-casual titles** to the App Store in **12 months**, each taken from core mechanic to full polish in 2–3 weeks: mechanic, UI, levels, SDK integration and store submission.
- Built a **shared, reusable toolkit** that carried from title to title and grew with every release; mechanics written for one game regularly became the foundation of the next.
- Cycles that short force scope and **game-feel** decisions in days rather than months: tune the controls, timing and juice until the mechanic feels right, cut everything that does not serve it.

Game Developer — Imaginite Studios

10/2019 – 04/2020

Kral Shakir İstanbul (licensed IP title) · Adana, Türkiye · [Video](#)

- Maintained and extended existing gameplay and UI systems on a licensed IP title.

Projects

Independent Game Developer

07/2026 – Present

Personal and open-source projects · İstanbul, Türkiye

- Cube Roll Puzzle** — my own mechanic, designed from zero and built as a full simulated studio pipeline: prototype, marketability test, tested pure-C# core, then foundation through live-ops, every milestone gated on a written KPI decision. [GitHub](#)

- **SmashFest Clone** — a trending mobile title rebuilt in 2–3 days: closed-form ballistic solver, **pure C# core** behind assembly definitions, **186 EditMode tests**, and a multi-layer grid level editor. [GitHub](#)
- **Nut Sort × Ball Sort hybrid** — one logic and data layer driving two completely different view layers, demonstrating **view-agnostic architecture**. [LinkedIn](#)
- **FluidSort** — my own mechanic, designed from zero out of market trend analysis. [itch.io](#) · [LinkedIn](#)
- **Buildris** — Tetris with a runtime tetromino level editor. [itch.io](#) · [LinkedIn](#)
- **Arrows replica** — reverse-engineered a trending mechanic to study its level design and feel. [LinkedIn](#)

Skills

Engine & Language: Unity, C#, Java, URP, Input System

Architecture: OOP, design patterns, SOLID, clean code, software architecture & scalable system design, Logic–Data–View separation, dependency injection (vContainer), assembly definitions, data-oriented design

Testing: Unity Test Framework, NUnit, EditMode & PlayMode tests, hand-written fakes, interface seams for non-deterministic code

Systems: Addressables, UniTask, Remote Config & A/B testing, analytics pipelines, save & profile data, audio, DOTween

Backend & Networking: FishNet, Mirror, client–server communication, offline/online state sync, physics-based multiplayer

Performance: Multithreading, index & vertex buffers, object pooling, allocation & GC profiling

Tooling & Release: Custom Unity editor tools, JSON-driven level pipelines, Google Play SDK, ad integration, Gradle, AAB builds & Play Console releases, Git

Practice: Rapid prototyping, live-ops, refactoring legacy codebases, code review, teamwork with Product, Art, QA and data teams, problem solving, clear communication, feature ownership, AI-assisted development under authored rule sets

Education

B.Sc. Computer Engineering — Erciyes University

Kayseri, Türkiye · CGPA 3.21 / 4.0

09/2014 – 12/2019